INFORME TECNICO

CobrApp

RESUMEN EJECUTIVO

CobrApp es un sistema automatizado creado para ayudar a las pymes a gestionar de forma más eficiente los comprobantes de pago digitales enviados por aplicaciones como Yape y Plin. El sistema recibe los comprobantes por Telegram, analiza las imágenes usando OCR, registra automáticamente los pagos en Google Sheets y envía una confirmación al cliente. Además, el informe explica la arquitectura del sistema, las tecnologías utilizadas y las soluciones aplicadas durante el desarrollo.

1. INTRODUCCION

1.1 Contexto y Problematica

En Perú, el uso de billeteras digitales como Yape y Plin ha crecido rápidamente, pero muchos pequeños negocios todavía verifican los pagos de manera manual. Los clientes envían capturas de sus comprobantes por WhatsApp o Telegram y los trabajadores deben revisar y registrar cada pago manualmente. Esto puede causar errores, registros duplicados y demoras en la atención. CobrApp nace como una solución de software para automatizar este proceso y hacerlo más rápido, ordenado y seguro.

1.2 Objetivos del Proyecto

Objetivo General

Diseñar, desarrollar e implementar un sistema automatizado de recepcion, validacion y registro de comprobantes de pago digitales, utilizando inteligencia artificial y flujos de automatizacion de bajo codigo.

Objetivos Especificos

- Automatizar la extraccion de datos clave (monto, numero de operacion, tipo de pago) desde imagenes de comprobantes mediante OCR.
- Implementar un mecanismo de validacion que distinga comprobantes validos de imagenes irrelevantes.

- Registrar los pagos validados de forma automatica en una base de datos accesible (Google Sheets).
- Generar y enviar reportes diarios consolidados al administrador del negocio.
- Construir el sistema sobre una arquitectura escalable, dockerizable y de costo operativo minimo.

1.3 Alcance

El sistema opera como un canal de atencion autonomo a traves de Telegram, procesando comprobantes de las plataformas Yape y Plin. La arquitectura esta optimizada para despliegue local con capacidad de migracion futura a infraestructura cloud (VPS en AWS EC2 o equivalente).

2. ARQUITECTURA DEL SISTEMA

2.1 Vision General

CobrApp sigue un patron de arquitectura de microservicios desacoplados, donde cada componente tiene una responsabilidad unica y bien definida. La comunicacion entre componentes se realiza a traves de llamadas HTTP/REST estandar, lo que garantiza la independencia tecnologica de cada capa y facilita el mantenimiento y la escalabilidad horizontal del sistema.

2.2 Stack Tecnologico

La siguiente tabla presenta el inventario completo de tecnologias que conforman el sistema CobrApp:

Componente	Tecnologia / Herramienta	Rol en el Sistema
Frontend / UI	`Bot de Telegram`	Canal de comunicacion directo con el cliente. Recibe las imagenes de los comprobantes y entrega las respuestas de confirmacion o error en lenguaje natural.
Orquestador de Flujos	`n8n (Docker)`	Motor de automatizacion de flujos de trabajo (workflow engine). Coordina la secuencia logica de pasos: recepcion de imagen, llamada a la API, evaluacion condicional y registro en base de datos.
Tunel HTTP	`ngrok`	Herramienta de tunel inverso que expone el servidor de webhooks local de n8n hacia los servidores de la API de Telegram, habilitando la comunicacion en entornos de desarrollo sin IP publica.
Backend / Motor de Procesamiento	`FastAPI + Uvicorn (Python)`	API REST de alto rendimiento que recibe la imagen, ejecuta el preprocesamiento con OpenCV, aplica el OCR con Tesseract, procesa el texto extraido con expresiones regulares y retorna un objeto JSON estructurado con los datos del comprobante.
Motor de IA (OCR)	`Tesseract OCR (spa)`	Motor de reconocimiento optico de caracteres de codigo abierto, configurado con el paquete de idioma espanol (tesseract-ocr-spa). Transforma el contenido visual de la imagen en texto plano procesable.

Preprocesamiento de Imagen	`OpenCV (cv2) + PIL`	Libreria de vision computacional que preprocesa la imagen antes de enviarla al OCR: convierte a escala de grises y aplica umbralizado binario para mejorar el contraste y la precision de la lectura.
Base de Datos	`Google Sheets (via n8n)`	Hoja de calculo en la nube utilizada como base de datos ligera. El nodo nativo de n8n gestiona la escritura de nuevas filas (pagos validos) y la lectura de datos para la generacion del reporte diario.

2.3 Diagrama de Arquitectura de Componentes

El siguiente esquema representa el flujo de informacion entre los componentes del sistema para el caso de uso principal (procesamiento de un comprobante en tiempo real):

```
[ Cliente ] ---(imagen)--> [ Bot Telegram ]
                                    |
                          [ Webhook (ngrok) ]
                                    |
                     [ n8n Workflow Engine ]
                              |          |
            [FastAPI + OCR]     [Google Sheets]
                              |
             [ Tesseract OCR + OpenCV ]
```

3. FLUJOS DE TRABAJO (WORKFLOWS)

El sistema CobrApp implementa dos flujos de trabajo independientes, cada uno diseñado para un proposito especifico y ejecutado bajo condiciones distintas de activacion.

3.1 Flujo 1: Procesamiento de Comprobantes en Tiempo Real

Descripcion General

Este flujo constituye el nucleo operativo del sistema. Se activa de forma reactiva cada vez que un cliente envia un mensaje al bot de Telegram. Su proposito es procesar la imagen recibida, extraer los datos del comprobante, validarlos y registrarlos, todo en cuestion de segundos.

Secuencia de Nodos

1. Telegram Trigger: Nodo de escucha activo que detecta los mensajes entrantes en el canal de Telegram configurado. Actua como punto de entrada del flujo y encapsula el objeto de mensaje completo (incluyendo metadatos del remitente e informacion del archivo adjunto) en la estructura de datos del workflow.
2. Accion 'Escribiendo...' (UX): Nodo de accion de Telegram que envia el indicador de estado de escritura. Esta decision de diseño mejora significativamente la experiencia del usuario al proporcionar retroalimentacion visual inmediata mientras el sistema procesa la imagen, evitando la percepcion de inactividad o error.
3. HTTP Request (POST a FastAPI): Nodo de solicitud HTTP que descarga la imagen del servidor de Telegram y la envia como una peticion POST al endpoint /procesar-imagen de la API de FastAPI, utilizando el formato multipart/form-data. El payload de respuesta es un objeto JSON estructurado con los campos extraidos y la bandera de validacion.
4. Nodo IF (Validacion Condicional): Nodo de bifurcacion logica que evalua el campo booleano valido presente en la respuesta JSON de FastAPI. Si el valor es true, el flujo continua hacia el registro en la base de datos. Si es false, el flujo se desvfa al nodo de error.
5. Google Sheets - Append Row: Nodo de escritura en Google Sheets que agrega una nueva fila al registro con los datos del pago validado: monto, numero de operacion, nombre del remitente, fecha de registro, tipo de pago (Yape/Plin) y texto crudo del OCR.
6. Telegram - Mensaje de Confirmacion: Nodo de respuesta que envia al cliente un mensaje personalizado de confirmacion, interpolando el nombre de pila del usuario extraido del objeto de mensaje original de Telegram mediante la expresion {{ $('Telegram Trigger1').item.json.message.from.first_name }}.

Rama: Pago VALIDO	**Rama: Pago INVALIDO**
Registro en Google Sheets + mensaje de exito al cliente con nombre personalizado.	Mensaje de error amigable al cliente indicando que la imagen no fue reconocida como un comprobante valido.

3.2 Flujo 2: Reporte Diario Automatizado

Descripcion General

Este flujo implementa un proceso de cierre contable diario automatico. Se ejecuta de forma programada y tiene como objetivo consolidar todos los pagos registrados durante la jornada y comunicarlos al administrador del negocio en un formato de resumen estructurado.

Secuencia de Nodos

7. Schedule Trigger: Nodo de activacion temporizada configurado para ejecutarse diariamente a las 23:59, utilizando la zona horaria America/Lima (UTC-05:00). Garantiza que el reporte capture la totalidad de transacciones del dia.
8. Google Sheets - Get Rows: Nodo de lectura que recupera todas las filas registradas en la hoja de calculo durante la jornada activa. El conjunto de datos resultante es pasado al siguiente nodo para su procesamiento.
9. Calculo de Suma Total: Nodo de codigo o expresion que itera sobre la coleccion de registros y calcula el monto total recaudado durante el dia, aplicando la suma aritmetica de todos los valores monetarios registrados.
10. Telegram - Envio de Reporte al Administrador: Nodo de mensajeria que formatea y envia al canal del administrador un resumen ejecutivo de la caja diaria, incluyendo el numero total de transacciones, el monto total recaudado y la fecha del reporte.

4. IMPLEMENTACION DEL BACKEND (FastAPI)

4.1 Arquitectura de la API

La API de procesamiento fue desarrollada con el framework FastAPI sobre Python, sirviendo las peticiones mediante el servidor ASGI Uvicorn. El diseño sigue el principio de responsabilidad unica (Single Responsibility Principle), separando las funciones de preprocesamiento de imagen, extraccion de texto y analisis de patron en funciones independientes.

4.2 Endpoints Implementados

Metodo	Endpoint	Descripcion
`POST`	`/procesar-imagen`	Endpoint principal. Recibe una imagen (multipart/form-data), ejecuta el pipeline OCR completo y retorna un JSON con los datos extraidos y el indicador de validez.
`GET`	`/pagos`	Retorna el listado completo de pagos registrados en el archivo de persistencia local (pagos.json), incluyendo el total de registros.
`GET`	`/reporte`	Genera y retorna un reporte de cierre con el total monetario recaudado, calculado dinamicamente a partir de la suma de todos los montos registrados en pagos.json.

4.3 Pipeline de Procesamiento OCR

Cada imagen recibida atraviesa el siguiente pipeline de procesamiento antes de la extraccion de texto:

11. Recepcion del archivo binario como UploadFile mediante FastAPI.
12. Conversion del buffer de bytes a un array NumPy compatible con OpenCV.
13. Conversion de la imagen a escala de grises (cv2.COLOR_BGR2GRAY) para reducir la complejidad de procesamiento.
14. Aplicacion de umbralizado binario (cv2.THRESH_BINARY, umbral=150) para mejorar el contraste entre el texto y el fondo.
15. Ejecucion de Tesseract OCR sobre la imagen preprocesada con el parametro de idioma espanol (lang='spa').
16. Analisis del texto crudo mediante expresiones regulares para la extraccion de campos especificos.
17. Construccion del objeto de respuesta JSON con todos los datos extraidos y el flag de validacion.

4.4 Estructura del Objeto de Respuesta JSON

```
{ "monto": "80.00", "operacion": "123456789", "nombre": "Juan Perez", "fecha":
"15/05/2025",
  "tipo": "Yape", "texto_raw": "...", "valido": true }
```

5. DIFICULTADES TECNICAS Y SOLUCIONES IMPLEMENTADAS

El proceso de desarrollo de CobrApp implico la identificacion y resolucion de cuatro incidencias tecnicas significativas. Cada una de estas dificultades requirio un analisis sistematico del problema y la implementacion de una solucion de ingenieria especifica. A continuacion se documenta cada caso:

Incidencia Tecnica N.1	
PROBLEMA IDENTIFICADO Baja Precision del OCR en la Lectura de Montos Monetarios. Tesseract OCR confundia sistematicamente el simbolo monetario 'S/.' (sol peruano) con el digito '5', y fallaba en la lectura correcta de los decimales separados por punto o coma (ej: '80.00' se leia como '8000'). Igualmente, los numeros de operacion, compuestos por largas cadenas de digitos, eran interpretados con errores de sustitucion de caracteres.	**SOLUCION IMPLEMENTADA** Implementacion de una capa de postprocesamiento de texto basada en Expresiones Regulares (Regex) en Python. Se diseñaron patrones de busqueda que no dependen de la lectura exacta del simbolo de moneda, sino que buscan la estructura numerica que le sigue: re.search(r'[sS5][/\\]?\s*(\d+(?:[.,]\d+)?)', texto). Este enfoque hace al sistema tolerante a los errores de sustitucion del OCR, extrayendo el valor numerico sin importar si el simbolo fue interpretado como 'S/', 'S', '5/' o '5'.

Incidencia Tecnica N.2	
PROBLEMA IDENTIFICADO Registro de Datos Nulos o Invalidos en la Base de Datos. Cuando un usuario enviaba una imagen que no correspondia a un comprobante de pago (una fotografia borrosa, un meme o una imagen sin texto relevante), el sistema procesaba la imagen, obtenia un texto vacio o sin los patrones buscados, y generaba una fila con campos nulos en Google Sheets, corrompiendo la integridad del registro.	**SOLUCION IMPLEMENTADA** Diseño de un sistema de validacion mediante una variable booleana centinela denominada 'valido'. La API de FastAPI evalua si el campo 'monto' fue exitosamente extraido; si el valor es None, establece valido = False. En el workflow de n8n, un Nodo IF evalua esta bandera antes de cualquier operacion de escritura: la rama 'true' ejecuta el registro en Google Sheets, mientras que la rama 'false' redirige el flujo hacia un nodo de Telegram que envia un mensaje de error amigable al usuario.

Incidencia Tecnica N.3	
PROBLEMA IDENTIFICADO Perdida de Contexto de Variables entre Nodos de n8n (Error: 'Can't get data for expression'). Al intentar personalizar el mensaje de confirmacion con el nombre del usuario (campo first_name del objeto de Telegram), el nodo final del workflow arrojaba	**SOLUCION IMPLEMENTADA** Correccion del mapeo de variables mediante el uso de referencias absolutas al nodo de origen utilizando la sintaxis de JavaScript nativa de n8n. En lugar de referenciar el nodo previo con $json o $node, se utilizo la expresion $('Telegram

el error de ejecucion 'Can't get data for expression'. Este error se originaba porque n8n, por defecto, solo expone los datos del nodo inmediatamente anterior en la cadena de ejecucion, perdiendo el acceso a los datos del nodo de origen tras pasar por multiples transformaciones intermedias.	Trigger1').item.json.message.from.first_name , que accede directamente al nodo de activacion inicial de forma independiente a la posicion actual en el flujo, garantizando la disponibilidad del dato en cualquier punto del workflow.

Incidencia Tecnica N.4	
PROBLEMA IDENTIFICADO Desfase Horario en la Ejecucion del Reporte Diario. El Schedule Trigger del Flujo 2 se ejecutaba a una hora incorrecta (con una diferencia de 5 horas respecto a la hora local de Lima, Peru). El sistema registraba transacciones del dia siguiente en el reporte del dia anterior, generando inconsistencias en el cierre contable.	**SOLUCION IMPLEMENTADA** Configuracion explicita de la zona horaria del flujo en n8n a America/Lima (UTC-05:00). La configuracion por defecto de los contenedores Docker de n8n utiliza UTC como zona horaria base. Al definir explicitamente la timezone del workflow a la zona horaria del negocio, el Schedule Trigger se sincronizo correctamente con la hora local, garantizando que el reporte se genere exactamente a las 23:59 hora Lima, capturando la totalidad de las transacciones de la jornada.

6. DECISIONES DE DISEÑO TECNICO

Las decisiones de arquitectura adoptadas en CobrApp responden a criterios de ingenieria de software bien fundamentados. A continuacion se justifica cada eleccion tecnologica clave:

6.1 FastAPI como Framework de Backend

La eleccion de FastAPI sobre alternativas como Flask o Django REST Framework se fundamenta en los siguientes criterios tecnicos:

- Alto rendimiento asincrono: FastAPI esta construido sobre Starlette y utiliza Python asyncio nativo, lo que le permite manejar multiples peticiones concurrentes sin bloqueo de hilo (I/O non-blocking), critico para un sistema que puede recibir imagenes de multiples usuarios simultaneamente.
- Generacion automatica de documentacion interactiva: FastAPI genera automaticamente la interfaz de documentacion Swagger UI en /docs y ReDoc en /redoc, facilitando el proceso de prueba y depuracion del API durante el desarrollo.
- Validacion de datos integrada: A traves de Pydantic, FastAPI valida automaticamente los tipos de datos de las peticiones y respuestas, reduciendo el codigo de validacion manual.
- Soporte nativo para archivos multipart: El parametro UploadFile de FastAPI gestiona de forma nativa la recepcion de archivos binarios (imagenes) enviados en formato multipart/form-data, que es el formato utilizado por n8n para transferir las imagenes.

6.2 Tesseract OCR como Motor de Inteligencia Artificial

La seleccion de Tesseract OCR sobre APIs de vision computacional de terceros (Google Cloud Vision, AWS Textract, Microsoft Azure Computer Vision) obedece a:

- Costo operativo cero: Tesseract es software libre bajo licencia Apache 2.0. Su uso no genera costos variables por volumen de peticiones, lo que es critico para la viabilidad economica de un sistema destinado a pymes.
- Control total del pipeline: Al ejecutarse localmente, el preprocesamiento de imagen (umbralizado, escala de grises) puede ser ajustado y optimizado sin dependencia de proveedores externos.
- Privacidad de los datos: Las imagenes de los comprobantes no son transmitidas a servidores de terceros, garantizando la confidencialidad de la informacion financiera del negocio y sus clientes.
- Soporte de idioma espanol: El paquete tesseract-ocr-spa esta entrenado especificamente para el reconocimiento de texto en español, mejorando la precision en comprobantes emitidos por Yape y Plin.

6.3 Arquitectura Dockerizable

El proyecto fue estructurado desde su concepcion con soporte para contenedorizacion mediante Docker:

- Reproducibilidad del entorno: El Dockerfile encapsula todas las dependencias del sistema (Python, Tesseract, OpenCV, librerias de Python) garantizando que el comportamiento del sistema sea identico en cualquier entorno de ejecucion.

- Facilidad de despliegue en la nube: Con un docker-compose.yml, la totalidad del sistema (FastAPI + n8n) puede ser desplegada en un VPS de AWS EC2 (capa gratuita con Ubuntu) o Digital Ocean con un unico comando, sin configuracion manual de dependencias.
- Escalabilidad horizontal: La arquitectura de contenedores permite replicar el servicio de FastAPI detras de un balanceador de carga si el volumen de transacciones lo requiere.

6.4 Google Sheets como Base de Datos

La eleccion de Google Sheets como capa de persistencia, en lugar de una base de datos relacional tradicional (PostgreSQL, MySQL), responde a un criterio pragmatico para el contexto de uso:

- Accesibilidad para el usuario final: El administrador del negocio puede visualizar, filtrar y exportar los registros de pagos desde cualquier dispositivo sin necesidad de software especializado.
- Integracion nativa con n8n: El nodo de Google Sheets de n8n gestiona la autenticacion OAuth y las operaciones CRUD sin necesidad de codigo adicional.
- Costo cero: Google Sheets es gratuito dentro de los limites de uso estandar de Google Workspace, eliminando costos de infraestructura de base de datos.
- Escalabilidad planificada: La arquitectura permite sustituir Google Sheets por una base de datos PostgreSQL en una fase futura sin modificar el nucleo del backend, ya que n8n abstrae la capa de persistencia.

7. CONCLUSIONES

El desarrollo e implementacion de CobrApp demuestra, de forma concreta y medible, el potencial de la integracion de microservicios de bajo costo para resolver problemas operativos reales en el segmento de pequeñas y medianas empresas. Las conclusiones derivadas del proyecto se presentan a continuacion:

7.1 Viabilidad Tecnica de la Arquitectura de Microservicios para Pymes

La combinacion de n8n como orquestador de flujos y FastAPI como motor de procesamiento establece un modelo de arquitectura que es simultaneamente potente y economico. La separacion de responsabilidades entre el componente de automatizacion (n8n) y el componente de inteligencia artificial (FastAPI + Tesseract) permite que cada capa evolucione de forma independiente. Si en el futuro se requiere sustituir Tesseract por un modelo de vision mas avanzado, dicho cambio se limita exclusivamente al backend de Python, sin afectar los flujos de n8n ni la experiencia del usuario en Telegram.

7.2 El Postprocesamiento es Tan Critico Como el OCR

Una de las lecciones tecnicas mas importantes del proyecto es que la precision de un sistema OCR no depende exclusivamente de la calidad del motor de reconocimiento, sino de la sofisticacion de la capa de postprocesamiento. La implementacion de expresiones regulares tolerantes a errores de sustitucion de caracteres demostro ser mas robusta y economica que intentar mejorar la calidad de entrada de la imagen para alcanzar un OCR perfecto. Esta estrategia de 'postprocesamiento inteligente' es aplicable a cualquier proyecto de extraccion de informacion de documentos.

7.3 La Validacion de Entradas es un Requisito de Integridad del Sistema

La implementacion del mecanismo de validacion booleana (campo valido) y el nodo condicional en n8n demostraron ser esenciales para la integridad de la base de datos. Un sistema que registra datos sin validacion previa genera 'deuda de datos' que se acumula y deteriora la calidad de los reportes y analisis posteriores. La decision de implementar validacion en la capa del backend (FastAPI) y reforzarla en la capa de orquestacion (n8n) crea un mecanismo de doble verificacion que garantiza que solo datos validos y completos lleguen a la base de datos.

7.4 Escalabilidad y Proyeccion Futura

CobrApp fue diseñado con una vision de escalabilidad que trasciende su implementacion inicial. La arquitectura dockerizable permite una migracion transparente desde un entorno de desarrollo local hacia infraestructura cloud (AWS EC2, Google Cloud Run, DigitalOcean Droplets). En una fase de expansion, el sistema puede ser extendido con:

- Un modelo de vision computacional entrenado especificamente sobre comprobantes Yape/Plin para mejorar la precision del OCR.
- Un dashboard web en tiempo real (React o Vue.js) que consuma los endpoints /pagos y /reporte de la API.
- Una base de datos relacional (PostgreSQL) para sustituir Google Sheets en escenarios de alto volumen transaccional.
- Integracion con sistemas de facturacion electronica (SUNAT) para la emision automatica de comprobantes.

7.5 ConclusionFinal

CobrApp demuestra cómo la ingeniería de software y las tecnologías open source pueden convertir procesos manuales y propensos a errores en sistemas automatizados, organizados y escalables. Su arquitectura, basada en Telegram, n8n, FastAPI, Tesseract OCR y Google Sheets, ofrece una solución económica, flexible y eficiente para las necesidades de las pymes, validando el uso de microservicios en este tipo de negocios.

8. REFERENCIAS TECNICAS

- FastAPI Documentation. (2025). FastAPI - Modern Python Web Framework. https://fastapi.tiangolo.com
- n8n Documentation. (2025). n8n - Workflow Automation Tool. https://docs.n8n.io
- Tesseract OCR. (2025). Tesseract Open Source OCR Engine. https://github.com/tesseract-ocr/tesseract
- OpenCV Documentation. (2025). Open Source Computer Vision Library. https://docs.opencv.org
- Telegram Bot API. (2025). Telegram Bot API Reference. https://core.telegram.org/bots/api
- Docker Documentation. (2025). Docker - Containerization Platform. https://docs.docker.com
- ngrok Documentation. (2025). ngrok - Secure Introspectable Tunnels. https://ngrok.com/docs
- Google Sheets API. (2025). Google Sheets API v4 Reference. https://developers.google.com/sheets/api